

Costo de API · Selección de modelo · Context window · Prompt caching · Monitoring · Escala

9

GUÍA 2 — MÓDULO 9

Optimización y Costos

Cómo escalar tu agente sin que los costos escalen igual

I — El costo que nadie calcula hasta que llega la factura

Un agente que funciona bien con 3 clientes puede volverse económicamente insostenible con 15. No porque el modelo sea caro en sí — sino porque nadie optimizó el uso. El contexto crece sin control, el modelo procesa tokens innecesarios en cada turno, y el costo por conversación sube de manera silenciosa.

Problema	Cómo se ve en la factura	Causa real
Context window sin límite	A medida que las conversaciones avanzan, cada turno procesa todo el historial anterior. Una conversación de 20 turnos puede costar 10x más que una de 2.	El agente nunca descarta contexto. Los tokens de entrada se acumulan turno a turno.
Modelo premium para todo	Se usa Claude Opus o GPT-4o para responder '¿cuáles son sus horarios?'	No hay distinción entre tareas simples y complejas. Un solo modelo para todo.
SOUL.md muy largo	El system prompt se envía completo en cada llamada a la API, aunque el 80% no sea relevante para esa conversación.	Nadie optimizó el SOUL.md después de escribirlo. Creció sin control.
Sin límites de uso por cliente	Un cliente con conversaciones muy largas consume la misma API que 5 clientes normales.	No hay max_tokens ni límites de sesión configurados.
Retries sin backoff	Cuando la API tiene timeout, el Gateway reintenta inmediatamente. Múltiples llamadas fallidas cobran tokens parciales.	Configuración default de reintentos sin estrategia de backoff exponencial.

→ El dato que cambia la perspectiva

GitHub Copilot costaba a Microsoft un promedio de USD 80/usuario/mes en usuarios intensivos en 2023, con precio de venta de USD 10/mes. La diferencia entre un agente rentable y uno que pierde plata no es el modelo que usás — es si controlás el consumo o no. La optimización no es reducir calidad. Es no pagar por tokens que no aportan valor.

II — El modelo correcto para cada tarea

El error más costoso es usar un modelo de alta capacidad para todo. La regla práctica: el modelo más caro se reserva para donde el razonamiento complejo cambia el resultado. Para el resto — FAQs, clasificación, respuestas estructuradas — los modelos económicos de 2026 dan resultados equivalentes a una fracción del costo. Y los modelos chinos como Qwen3 o DeepSeek-V3 cambiaron completamente la ecuación de costo para tareas internas.

Tipo de tarea	Modelos recomendados (2026)	Costo aproximado	Por qué alcanza
FAQs, horarios, precios, derivación simple	Claude Haiku 4.5 · GPT-4o-mini · Gemini 2.0 Flash · Grok 3 Mini	USD 0,10–0,40/M tokens	Preguntas con respuesta predecible. Alto volumen, bajo costo, calidad suficiente.
Clasificación, routing, extracción interna de datos	Qwen3-8B / Qwen3-14B (Ollama, local) · DeepSeek-V3 · Gemini Flash	Casi cero si local; USD 0,05–0,20/M si API	No interactúan directamente con el usuario. Modelos chinos económicos rinden muy bien en estas tareas.
Calificación de leads, objeciones, conversaciones con contexto	Claude Sonnet 4.6 · GPT-4o · Grok 3 · Gemini 2.5 Pro	USD 2–8/M tokens	Necesitan comprensión de matices pero no razonamiento de cadena larga. Grok es especialmente rápido.
Análisis complejo, redacción estratégica, decisiones multi-variable	Claude Opus 4.6 · GPT-5.2 / 5.3 · Gemini 3.1 Pro	USD 10–60/M tokens	Tareas donde la calidad del razonamiento impacta directamente el resultado del negocio.
Código complejo, debugging, razonamiento técnico extendido	Claude Opus 4.6 · GPT-5.4 · Grok 4	USD 15–75/M tokens	Modelos con ventanas de contexto largas y capacidad de razonamiento multi-paso real.

Configurar fallbacks por modelo en openclaw.json

```
# Agente con modelo económico como primario y fallback a uno más potente:
"agents": {
  "list": [
    {
      "id": "agente-soporte",
      "model": {
        "primary": "anthropic/claude-haiku-4-5",
        "fallbacks": [
          "anthropic/claude-sonnet-4-6",
          "openai/gpt-4o-mini"
        ]
      }
    ]
  }
}
```

```
}  
]  
}  
  
# El Gateway usa el primario por defecto.  
# Si hay error o timeout, pasa automáticamente al siguiente fallback.  
# Nunca queda sin respuesta por un fallo del proveedor.
```

⚠ **Los modelos económicos tienen límites reales que hay que conocer**

Claude Haiku 4.5, GPT-4o-mini, Gemini Flash y Grok 3 Mini son excelentes para tareas simples pero pueden degradar calidad en conversaciones largas o con SOUL.md muy complejos. Si el agente empieza a dar respuestas extrañas después de varios turnos, el primer sospechoso es la combinación modelo económico + historial largo.

Los modelos chinos (Qwen3, DeepSeek) brillan en tareas estructuradas pero pueden tener matices distintos en español coloquial latinoamericano. Testar antes de producción con la variante regional de tus usuarios.

Context window — el gasto que crece en silencio

III — Context window: controlar cuánto historial procesa el modelo

Cada vez que el agente responde, el modelo recibe todo el historial de la conversación desde el primer mensaje. En el turno 2 procesa 2 mensajes. En el turno 15 procesa 15 mensajes. El costo de tokens de entrada crece linealmente con cada turno — y en conversaciones largas puede ser el 70-80% de la factura total.

Opciones de gestión de contexto en OpenClaw

Mecanismo	Qué hace	Cuándo usar
Auto-compaction	OpenClaw resume automáticamente el historial cuando se acerca al límite del context window. El resumen reemplaza los mensajes originales.	Activado por defecto. Es el mecanismo más transparente — el agente mantiene continuidad sin que el usuario lo note.
maxContextTokens	Define un techo explícito de tokens de contexto por sesión. Al alcanzarlo, OpenClaw aplica compaction o trunca el historial más	Para agentes con conversaciones muy largas donde querés control preciso del gasto.

antiguo.

Sesiones cortas
por diseño

El agente está configurado para resolver en pocos turnos y cerrar la sesión. /new inicia una sesión limpia.

Agentes de soporte o FAQ donde cada consulta es independiente. No acumulan contexto entre consultas.

summaryOnClose

Al cerrar una sesión, el agente escribe un resumen compacto en MEMORY.md. La próxima sesión arranca desde ese resumen, no desde el historial completo.

Para agentes con contexto de usuario persistente (ventas, seguimiento) donde querés continuidad sin costo de historial largo.

Configurar límites de contexto

```
# En openclaw.json — límite de tokens de contexto por agente:
"agents": {
  "defaults": {
    "context": {
      "maxTokens": 8000,
      "compactionStrategy": "summarize",
      "summaryOnClose": true
    }
  }
}

# maxTokens: 8000 — techo de tokens de contexto (entrada + historial)
# compactionStrategy: summarize — resume en lugar de truncar
# summaryOnClose: true — genera resumen al cerrar sesión para MEMORY.md

# Para ver el consumo de tokens de una sesión activa:
$ openclaw sessions stats NOMBRE_SESION
```



El truco más efectivo: SOUL.md corto y preciso

El SOUL.md se envía completo en cada llamada a la API. Un SOUL.md de 3.000 tokens cuesta el triple que uno de 1.000 tokens en cada turno de cada conversación de cada cliente. Revisalo periódicamente: eliminá redundancias, consolidá reglas similares, y quitá instrucciones que el agente ya no necesita.

La diferencia entre un SOUL.md de 800 tokens y uno de 3.000 tokens, con 500 conversaciones mensuales de 5 turnos cada una, puede ser USD 20-40/mes de diferencia pura — sin cambiar nada del modelo.

IV — Prompt caching: no pagar dos veces por el mismo texto

Anthropic y OpenAI ofrecen descuentos automáticos sobre tokens que se repiten entre llamadas consecutivas. Cuando el prefijo de un prompt (el system prompt + contexto fijo) es idéntico en múltiples llamadas, los proveedores aplican un precio reducido — hasta 90% menos en algunos modelos — porque ya tienen ese texto procesado en caché.

Proveedor	Descuento en tokens cacheados	Condición para activarse	Aplica a
Anthropic (Claude)	Escritura de caché: +25% sobre precio base. Lectura de caché: −90% sobre precio base.	El prefijo debe tener al menos 1.024 tokens y repetirse en llamadas consecutivas dentro de una ventana de tiempo.	Principalmente el system prompt (SOUL.md). Cuanto más largo y estable, más ahorro.
OpenAI (GPT-4o / o4-mini / GPT-4.5)	−50% en tokens de entrada cacheados.	Prefijos de 1.024+ tokens que se repiten. Se activa automáticamente sin configuración explícita.	System prompt + cualquier contenido fijo al inicio del contexto.
OpenRouter	Depende del proveedor del modelo seleccionado.	Algunos modelos lo soportan, otros no. Verificar en la documentación del modelo específico.	Variable según modelo.

Cómo aprovechar el caching desde OpenClaw

El caching se activa automáticamente cuando el SOUL.md es suficientemente largo y estable. No requiere configuración adicional en OpenClaw — pero hay dos prácticas que maximizan el ahorro:

Práctica	Por qué funciona	Qué evitar
Mantener el SOUL.md estable entre sesiones	El caché se invalida si el prefijo cambia. Un SOUL.md que se edita frecuentemente no acumula beneficio de caché.	Modificar el SOUL.md más de una vez por semana en producción activa.
Poner el contenido fijo al inicio del SOUL.md	El caching funciona por prefijo. El contenido que no cambia debe estar primero — identidad, scope, reglas base — y el contenido dinámico (si existe) al final.	Intercalar instrucciones dinámicas entre las reglas base del SOUL.md.

Límites de gasto — nunca una sorpresa en la factura

V — Controlar el gasto antes de que se escape

Los dashboards de Anthropic y OpenAI permiten configurar alertas y límites de gasto. Combinados con los límites de tokens de OpenClaw, forman una doble capa de protección. Ninguno de los dos solo es suficiente — los límites de proveedor cortan el servicio, los de OpenClaw gestionan elegantemente.

Capa 1 — Límites en el proveedor de modelo

Proveedor	Dónde configurar	Qué configurar
Anthropic	console.anthropic.com → Settings → Limits	Soft limit (alerta por email) y Hard limit (corta el servicio). Ambos en USD/mes.
OpenAI	platform.openai.com → Settings → Limits	Monthly budget (alerta) y Hard limit (corta). También limit por clave de API específica.
OpenRouter	openrouter.ai → Settings → Usage	Límite de crédito disponible. Se recarga manualmente — sin crédito, sin servicio.

Capa 2 — Límites por sesión en OpenClaw

```
# Límites de tokens y costo máximo por sesión:
"agents": {
  "defaults": {
    "limits": {
      "maxTokensPerTurn": 1500,
      "maxTokensPerSession": 20000,
      "maxCostPerDay": 2.00
    }
  }
}

# maxTokensPerTurn: 1500 — el agente no genera respuestas más largas que esto
# maxTokensPerSession: 20000 — techo de tokens totales por sesión de usuario
# maxCostPerDay: 2.00 — si el agente llega a USD 2 en un día, se pausa

# Para ver el gasto actual del día:
$ openclaw stats --today

# Para ver el gasto por agente en el último mes:
$ openclaw stats --agent agente-soporte --month
```

⚠ Un cliente con conversaciones muy largas puede consumir el presupuesto de 10

Sin maxTokensPerSession, un usuario que deja una conversación abierta todo el día y sigue escribiendo puede acumular un costo que no tiene ninguna relación con el valor que genera. Este límite no corta la experiencia — simplemente inicia una sesión nueva cuando se

alcanza, con un mensaje configurado en el SOUL.md.

VI — Monitoring: saber qué pasa cuando no estás mirando

Un agente en producción sin monitoring es un agente que falla en silencio. No hace falta una plataforma de observabilidad empresarial — con los logs nativos de OpenClaw y alertas básicas se puede detectar el 90% de los problemas antes de que afecten a usuarios.

Las métricas que importan — y cómo verlas

Métrica	Por qué importa	Cómo verla en OpenClaw
Costo por día / por agente	Detecta picos anómalos antes de que llegue la factura mensual.	<code>openclaw stats --today / --week --agent NOMBRE</code>
Tokens promedio por turno	Un aumento sostenido indica que el SOUL.md creció o que las conversaciones se están alargando sin razón.	<code>openclaw stats --tokens --last 7d</code>
Tasa de error de API	Errores frecuentes indican problemas con el proveedor, límites alcanzados, o prompts que exceden el context window.	<code>cat /tmp/openclaw/openclaw-FECHA.log grep -i error wc -l</code>
Sesiones activas simultáneas	Un pico inusual puede indicar un loop de conversación, un ataque de spam, o un usuario que scriptea el bot.	<code>openclaw sessions list --active</code>
Tiempo de respuesta promedio	Respuestas lentas indican saturación del VPS o throttling del proveedor.	<code>openclaw stats --latency --last 24h</code>

Alerta básica por email — sin herramientas externas

```
# Script simple de monitoreo diario — agregar al cron del VPS:
# crontab -e → agregar la línea:
```

```
$ 0 9 * * * /root/check_openclaw.sh

# Contenido del script check_openclaw.sh:
#!/bin/bash
$ ERRORS=$(cat /tmp/openclaw/openclaw-$(date +%Y-%m-%d).log | grep -i error | wc -l)
$ COST=$(openclaw stats --today --json | jq .cost_usd)

$ if [ "$ERRORS" -gt 20 ] || (( $(echo "$COST > 3.0" | bc -l) )); then
  echo "Alerta OpenClaw: $ERRORS errores, costo del día: $COST USD" | mail -s "Alerta
agente" tu@email.com
$ fi

# Esto envía un email solo cuando hay algo que atender.
# Sin alertas = todo normal.
```

→ **El dashboard más útil es el que revisás todos los días**

Las herramientas de observabilidad avanzada (Datadog, Grafana, LangSmith) son valiosas a escala. Con 1-10 clientes, el cron diario más los comandos nativos de OpenClaw cubren todo lo necesario. La complejidad de infraestructura de monitoring debe crecer con el negocio, no adelantarse a él.

VII — Checklist de optimización por etapa

No todo aplica desde el día 1. Este checklist está ordenado por prioridad según la cantidad de clientes activos.

Etapa	Qué hacer primero	Impacto esperado
1-3 clientes	Configurar soft y hard limits en el dashboard del proveedor. Verificar que el SOUL.md no tiene contenido redundante.	Evita sorpresas. Bajo esfuerzo, alto valor de tranquilidad.
4-8 clientes	Evaluar si el modelo primario es el correcto para el tipo de tarea. Activar maxTokensPerSession. Revisar longitud promedio del SOUL.md.	Reducción de 20-40% en costo mensual en la mayoría de los casos.
9-15 clientes	Configurar modelos distintos por tipo de tarea o por agente. Activar summaryOnClose. Implementar el script de monitoreo diario.	El costo crece más despacio que el número de clientes.
15+ clientes	Analizar logs para identificar los patrones de conversación más costosos. Considerar modelo local (Ollama) para tareas de clasificación interna. Revisar si el caching está siendo efectivo.	Optimización de largo plazo. El margen del servicio se estabiliza.

Lo que reduce costo

Lo que no vale la pena optimizar

SOUL.md corto y sin redundancias	Micro-optimizar cada prompt individual
Modelo económico para tareas simples	Cambiar de proveedor cada vez que sale uno más barato
maxTokensPerSession activo	Obsesionarse con reducir latencia antes de tener volumen real
summaryOnClose para agentes con continuidad	Reescribir el agente desde cero para ahorrar tokens
Límites en el dashboard del proveedor	Implementar caching manual si el sistema ya lo hace automático

♦ Guía 2 completa

Con este módulo terminás el recorrido completo de la Guía 2 — desde Security hasta la optimización en producción. El paso siguiente es operar, iterar, y crecer con lo que construiste.